

# Robust Gamma-KalmaNet State Estimation for QCar

Quang Huy Nugyen

Echrak Chnib

June 6, 2026

## Abstract

This report summarizes the robust KalmaNet/RKNet estimator developed for the QCar platform. The estimator keeps the QCar physical prediction model and learns the uncertain part of the filter: how much each measurement should be trusted and how strongly the state should be corrected. In this report the learned measurement trust vector is denoted by  $\gamma_k$  and is treated as the “robust gamma” gate. The current implementation estimates  $\mathbf{x} = [x, y, \psi, v]^T$  from GPS position, fused heading, tachometer velocity, IMU, steering, and wheel speed signals. Training uses real QCar logs together with synthetic GPS, heading, velocity, IMU, steering, and wheel-speed attacks. The saved training history shows a large reduction in attacked validation loss: from 1.1095 at the first logged epoch to 0.4028 at the best robust checkpoint. Offline validation on a separate dataset also shows lower four-state RMSE for the learned estimator than the kinematic reference. The main conclusion is that the method is not a pure black-box neural estimator; it is a hybrid filter where vehicle physics gives the prior and the recurrent neural update learns when to reject corrupted measurements.

## 1 Introduction

Autonomous QCar experiments require a state estimate that is accurate enough for feedback control and stable enough during bad sensor periods. A standard Extended Kalman Filter (EKF) is a strong baseline when its assumptions are correct. It uses a vehicle model for prediction and a measurement model for correction. However, the EKF can become fragile when the measurement noise is not just random noise but a structured fault: GPS jumps, GPS dropout, frozen velocity, wheel speed scaling, heading glitches, or short IMU bursts. In those cases the filter may apply a correction in the wrong direction because the measurement is still treated as believable.

The work in this folder builds a robust KalmaNet-style estimator for the QCar. The key design choice is simple:

keep the vehicle model for prediction, and learn a trust-controlled update for difficult sensor behavior.

The learned trust vector  $\gamma_k$  is the important element. When a channel is clean, its trust should stay high. When a channel is attacked or unavailable, its trust should decrease so that the innovation cannot force a large state jump. This is close in spirit to KalmaNet [1] and RobustStateNet [2], but the current QCar implementation is more conservative: the prediction step is currently an analytical QCar bicycle model, not a learned predictor.

## 2 Problem Definition

### 2.1 State and Measurement

The active training and runtime state is

$$\mathbf{x}_k = [x_k \quad y_k \quad \psi_k \quad v_k]^T, \quad (1)$$

where  $x$  and  $y$  are global position,  $\psi$  is heading, and  $v$  is forward velocity. Some older recorded files contain a fifth yaw-rate channel, but the current robust KalmaNet configuration slices the target and measurement arrays to the first four states.

The measurement vector is

$$\mathbf{z}_k = [x_{gps,k} \quad y_{gps,k} \quad \psi_{fused,k} \quad v_{tach,k}]^T, \quad \mathbf{z}_k = H\mathbf{x}_k + \mathbf{n}_k, \quad H \approx I_4. \quad (2)$$

The measured heading is rebuilt offline using the same QCar heading-fusion path used by the local estimator. GPS metadata is kept because it is essential to separate real measurement faults from normal innovation.

## 2.2 Raw Sensor Inputs

The model receives raw inputs grouped by physical source:

- IMU branch: velocity context, heading context, acceleration, and yaw rate;
- steering branch: velocity context, heading context, and steering angle;
- wheel branch: velocity context, heading context, and four wheel-speed channels;
- availability metadata: GPS valid flag, heading valid flag, GPS hold flag, and GPS age.

This separation matters because each sensor family has different failure behavior. GPS position can jump or disappear. Velocity can freeze or become scaled. IMU signals can become noisy or biased. The learned update uses these signals to decide whether the current measurement should be trusted.

## 3 Estimator Architecture

Figure 1 shows the estimator at a high level. The prediction side is deterministic. The update side is learned and recurrent.

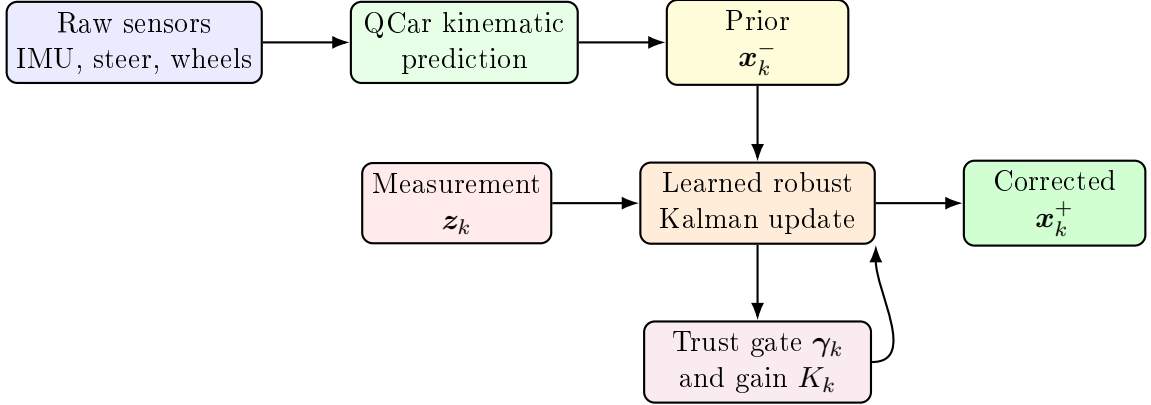


Figure 1: Hybrid RKNet structure used in this work. The prior is generated by QCar physics. The neural update learns a measurement trust gate and a Kalman-style gain.

### 3.1 Kinematic Prediction

The active configuration uses `predictor_mode: kinematic`. The prediction step has no learned parameters. It is a bicycle-model update with wheelbase  $L$ :

$$\dot{\psi}_k = \frac{v_k \tan(\delta_k)}{L}, \quad (3)$$

where  $\delta_k$  is the steering angle. The predicted pose is

$$x_k^- = x_{k-1}^+ + v_{pose,k} \cos(\psi_{k-1}^+) \Delta t, \quad (4)$$

$$y_k^- = y_{k-1}^+ + v_{pose,k} \sin(\psi_{k-1}^+) \Delta t, \quad (5)$$

$$\psi_k^- = \text{wrap} \left( \psi_{k-1}^+ + \frac{v_{pose,k} \tan(\delta_k)}{L} \Delta t \right). \quad (6)$$

The active velocity model is `velocity_lag`. It predicts velocity from the previous velocity and throttle:

$$\dot{v}_k = \frac{-v_{k-1}^+ + g u_k}{\tau}, \quad v_k^- = \text{clip}(v_{k-1}^+ + \dot{v}_k \Delta t). \quad (7)$$

The current values in `robust_kalmanet_train_config.yaml` are  $L = 0.2$ ,  $\tau = 0.301$ ,  $g = 6.598$ ,  $|v| \leq 2.0$  m/s, and maximum acceleration 2.0 m/s<sup>2</sup>.

### 3.2 Learned Robust Update

The learned update receives the prior  $\mathbf{x}_k^-$ , the previous corrected state  $\mathbf{x}_{k-1}^+$ , the current measurement  $\mathbf{z}_k$ , the previous measurement  $\mathbf{z}_{k-1}$ , and availability metadata. It forms three main residual terms:

$$\Delta \mathbf{x}_k = \text{wrap}(\mathbf{x}_k^- - \mathbf{x}_{k-1}^+), \quad (8)$$

$$\mathbf{r}_k = \text{wrap}(\mathbf{z}_k - H \mathbf{x}_k^-), \quad (9)$$

$$\Delta \mathbf{z}_k = \text{wrap}(\mathbf{z}_k - \mathbf{z}_{k-1}). \quad (10)$$

The hard availability vector is

$$\mathbf{a}_k = [g_k \quad g_k \quad h_k \quad 1]^T, \quad (11)$$

where  $g_k$  is the GPS-position valid flag and  $h_k$  is the heading valid flag. Unavailable measurement channels are removed before the learned correction.

The update feature vector is

$$\mathbf{f}_k = [\Delta \mathbf{x}_k / \mathbf{s}_x, (\mathbf{a}_k \odot \mathbf{r}_k) / \mathbf{s}_z, (\mathbf{a}_k \odot \Delta \mathbf{z}_k) / \mathbf{s}_z, \mathbf{a}_k, g_k^{hold}, \log(1 + \min(age_k, 10)), \log(1 + |\mathbf{a}_k \odot \mathbf{r}_k|), \log(1 + |\mathbf{a}_k \odot \Delta \mathbf{z}_k|)]. \quad (12)$$

For the four-state model this gives a 26-dimensional update feature vector. The scale vectors used by the active config are

$$\mathbf{s}_x = \mathbf{s}_z = [0.25, 0.25, 0.20, 0.50].$$

The robust gamma/trust vector is generated by a small MLP:

$$\boldsymbol{\gamma}_k = \sigma(\text{MLP}_\gamma(\mathbf{f}_k)). \quad (13)$$

Only the measurement part of this vector is used directly as the measurement trust gate. Clean channels should stay close to one; attacked channels should move toward zero.

The Kalman-style gain is generated by a GRU and an MLP:

$$\mathbf{h}_k = \text{GRU}(\boldsymbol{\gamma}_k \odot \mathbf{f}_k, \mathbf{h}_{k-1}), \quad (14)$$

$$K_k = s \tanh(\text{MLP}_K(\mathbf{h}_k)). \quad (15)$$

The current tanh scale is  $s = 1.2$ . The implementation also applies exponential smoothing to the gain and trust gate with  $\alpha = 0.35$ .

The corrected state is

$$\mathbf{x}_k^+ = \mathbf{x}_k^- + K_k (\mathbf{a}_k \odot \boldsymbol{\gamma}_k^z \odot \mathbf{r}_k), \quad (16)$$

where  $\boldsymbol{\gamma}_k^z$  is the measurement-trust part of the learned gate. Equation (16) is the main robustness mechanism. A corrupted GPS or velocity measurement may create a large innovation, but the innovation is first multiplied by the learned trust gate before it reaches the gain. The runtime also clamps the per-step correction to

$$[0.35, 0.35, 0.35, 0.80]$$

for  $[x, y, \psi, v]$ .

## 4 Dataset and Training

### 4.1 Recorded Dataset

The dataset recorder saves time-aligned raw signals, measurement vectors, target states, timestamps, and JSON metadata. The active training configuration points to

`datasets/robust_kalmanet_dataset_V0_20260417_094048.npz`.

Its metadata reports 52,921 samples, 671.84 seconds of driving, and mean timestep 0.0127 seconds. The target estimator type is EKF, so the supervised target is a clean/reference estimator rather than motion-capture ground truth.

Table 1: Main dataset and training configuration used by the saved model.

| Item                    | Value                                        |
|-------------------------|----------------------------------------------|
| State order             | $[x, y, \psi, v]$                            |
| Measurement order       | $[x_{gps}, y_{gps}, \psi_{fused}, v_{tach}]$ |
| Active dataset          | <code>V0_20260417_094048.npz</code>          |
| Samples / duration      | 52,921 samples / 671.84 s                    |
| Mean timestep           | 0.0127 s                                     |
| Validation split        | final 20% of sequence                        |
| Training windows        | $T = 50$ for Phase B, $T = 80$ for Phase C   |
| Predictor mode          | kinematic QCar bicycle model                 |
| Updater hidden size     | 32                                           |
| Gain hidden size        | 32                                           |
| Optimizer learning rate | $1.0 \times 10^{-4}$                         |

### 4.2 Training Phases

The code supports a learned predictor, but the active predictor is kinematic. Therefore predictor pretraining is automatically skipped. The useful phases are:

- Phase B: freeze the predictor and train only the learned updater;
- Phase C: fine tune the complete recurrent runtime loop with teacher forcing disabled.

The state loss is weighted MSE:

$$L_{state} = \frac{1}{N} \sum_k \left\| W^{1/2} (\hat{\mathbf{x}}_k - \mathbf{x}_k^*) \right\|_2^2, \quad W = [1.0, 1.0, 2.5, 1.2]. \quad (17)$$

The full loss combines state accuracy, trust supervision, gain regularization, and temporal smoothness:

$$\mathcal{L} = \lambda_u L_{upd} + \lambda_p L_{pred} + \lambda_\gamma L_\gamma + \lambda_K L_K + \lambda_{\Delta K} L_{\Delta K} + \lambda_{\Delta \gamma} L_{\Delta \gamma}. \quad (18)$$

The active Phase C weights are  $\lambda_u = 0.85$  and  $\lambda_p = 0.15$ . Measurement-mask supervision uses a clean target of one and an attacked target of zero for each affected measurement channel.

## 5 Sensor Attack Augmentation

The training data is not only clean driving. Synthetic attacks are injected offline so the model sees examples of sensor faults and receives supervision for which channels should be trusted.

The GPS attacks are especially important for QCar because GPS position directly drives  $x$  and  $y$  correction. A jump attack can make the EKF snap to a wrong position unless the measurement is down-weighted. A dropout attack is different: the availability gate should remove  $x/y$ , while the learned update continues using the remaining channels.

Table 2: Active augmentation configuration.

| Attack group                  | Probability | Enabled failure types                       |
|-------------------------------|-------------|---------------------------------------------|
| Raw branch attack             | 0.25        | bias, scale, freeze, noise, ramp, zero-out  |
| Direct measurement attack     | 0.35        | heading and velocity corruption             |
| GPS position attack           | 0.45        | noise, freeze, jump, dropout, reacquisition |
| Maximum raw branches attacked | 1           | one of IMU, steering, wheel                 |

## 6 Results

### 6.1 Training History

The saved file `models/robust_kalmanet.train_history.json` contains 20 logged epochs after Phase A was skipped. The best robust checkpoint was selected at epoch 15 in Phase B. Table 3 summarizes the main points.

Table 3: Saved training history. Selection loss combines clean and attacked validation, with attacked validation weighted more strongly.

| Checkpoint point         | Epoch | Clean val. | Attacked val. | Selection loss |
|--------------------------|-------|------------|---------------|----------------|
| First logged epoch       | 1     | 0.00668    | 1.10951       | 0.66838        |
| Best robust checkpoint   | 15    | 0.01751    | 0.40279       | 0.24868        |
| Final Phase C checkpoint | 20    | 0.02408    | 0.43457       | 0.27037        |

The attacked validation loss decreases from 1.10951 to 0.40279 at the selected best robust checkpoint, a reduction of about 63.7%. The clean validation loss is higher at the robust checkpoint than at the first clean-only epoch. This is an expected tradeoff: the estimator becomes more conservative so it can survive corrupted measurements.

### 6.2 Offline Validation

The validation script compares a kinematic reference and the learned RKNet checkpoint against the saved EKF target. The file `logs/validation/validation_metrics_best.json` reports the following result on `V0_20260401_170829.npz`. Older validation files contain an extra yaw-rate channel, so Table 4 reports only the active four-state surface.

Table 4: Offline RMSE against the EKF target on the best validation file.

| Method              | $x$ [m] | $y$ [m] | $\psi$ [rad] | $v$ [m/s] | Mean, four states |
|---------------------|---------|---------|--------------|-----------|-------------------|
| Kinematic reference | 0.0663  | 0.0574  | 0.1151       | 0.0134    | 0.0631            |
| Learned RKNet       | 0.0076  | 0.0168  | 0.1096       | 0.0124    | 0.0366            |

The learned estimator strongly improves  $x$  and  $y$  RMSE and gives a smaller four-state mean RMSE. Heading remains the hardest channel. This is reasonable because heading is affected by angle wrapping, GPS heading availability, and the reference itself.

### 6.3 Runtime Comparator Run

The latest plotted workspace summary used in this report is `rknet_workspace_20260503_025905`. It contains 8,413 model samples over 95.97 seconds, with no fallback samples. It includes five attack segments and 1,510 attack-active samples. Only 421 samples had fresh GPS, while 8,380 used GPS hold validity, so this run is useful for checking robustness during limited fresh GPS availability.

## Testing, Diagnostics, and Result

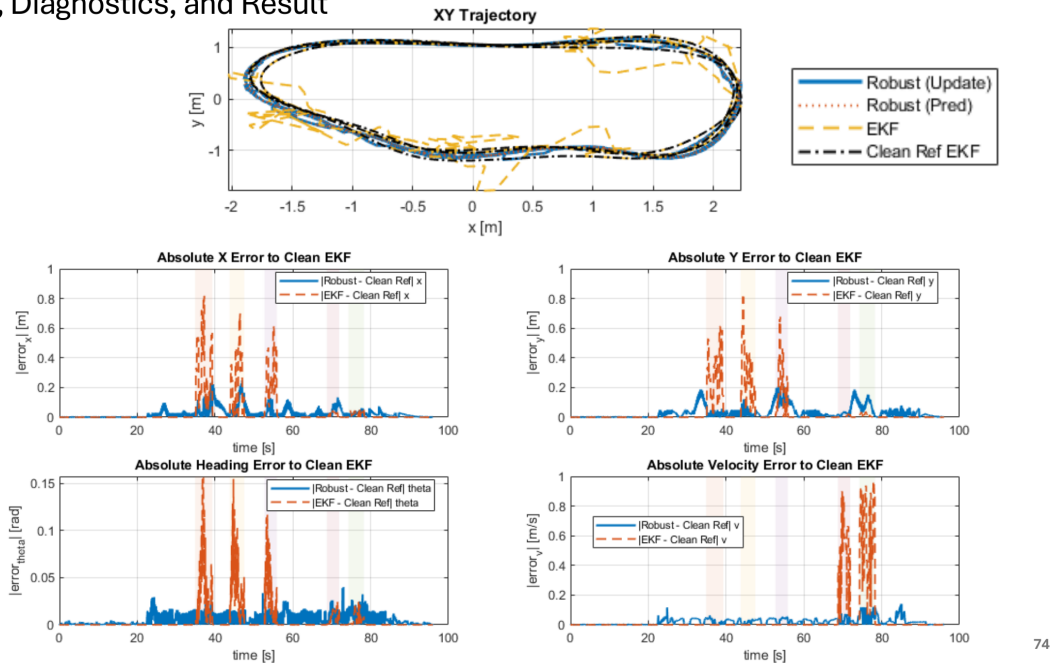


Figure 2: Result figure rendered from `KalmaRobust_Net_Huy.pdf`. The robust estimator follows the clean-reference EKF while the ordinary EKF shows larger spikes during attack intervals.

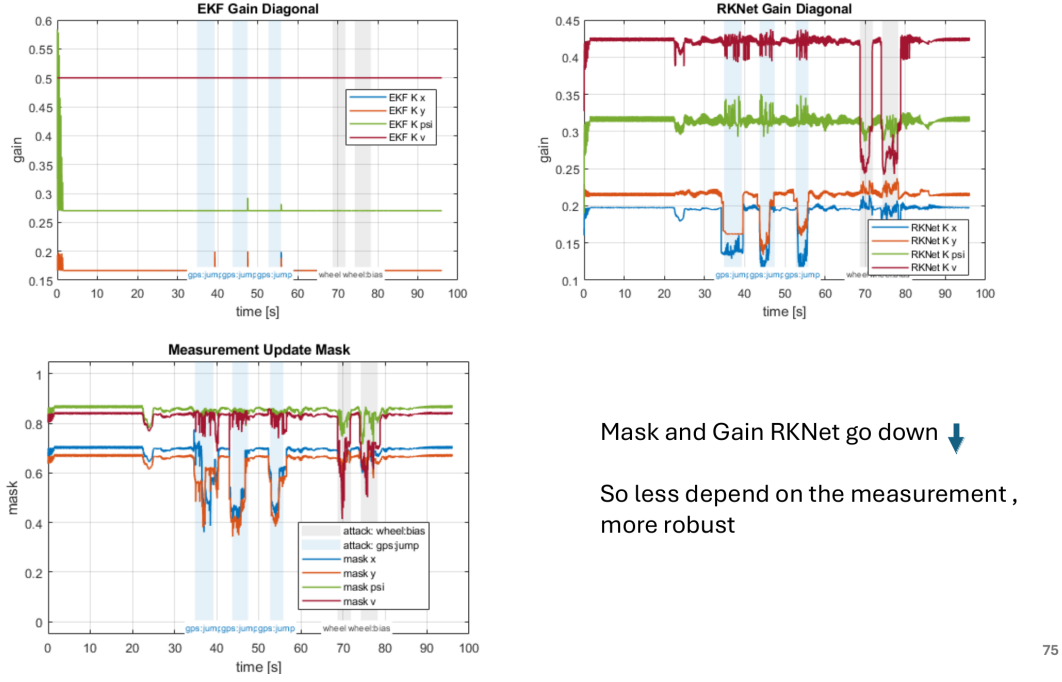


Figure 3: Gain and measurement-mask behavior rendered from KalmaRobust\_Net\_Huy.pdf. The learned mask and effective gain decrease during GPS jump and wheel-bias intervals, reducing dependence on corrupted measurements.

Figures 2 and 3 show the main behavior expected from the robust gamma gate. During the GPS jump intervals, the measurement mask for position decreases and the effective diagonal gain also drops. During wheel-bias intervals, the velocity-side trust and gain become more conservative. This means the update is not blindly following the measurement; it is using the innovation only after passing it through the learned reliability gate.

## 7 Discussion

The important engineering result is the separation of responsibilities. The QCar prediction step carries the vehicle kinematics and basic longitudinal behavior. The neural update does not need to relearn the full vehicle model. Instead, it learns the part that is hard to tune manually: the measurement trust policy under abnormal sensor conditions.

The learned trust mechanism also makes the estimator more interpretable than a single end-to-end recurrent network. The plotted  $\gamma_k$  values show when the estimator is reducing confidence in GPS position, heading, or velocity. The gain plots show whether the final correction is aggressive or conservative. This is useful for debugging because a bad output can be traced to three quantities: the prior state, the innovation, and the trust/gain pair.

There are also limitations:

- the target is an EKF/reference estimator, not absolute ground truth;
- the attacks are synthetic, so real hardware failures still need more dedicated experiments;
- the current active predictor is kinematic, so raw-branch learned feature masking is not used in the prediction path;
- the model is trained mainly on the available V0 QCar logs and should be rechecked before claiming generalization to other vehicles or tracks;
- the clean/robust tradeoff is visible: robustness improves attacked validation loss, while clean validation loss becomes slightly worse.

## 8 Conclusion

The robust gamma-KalmaNet estimator developed in this work is a practical hybrid state estimator for QCar. It preserves a deterministic QCar kinematic prediction model and learns a recurrent Kalman-style update with a measurement trust gate. The trust gate suppresses corrupted GPS, heading, and velocity channels before the innovation is applied to the state. Training with synthetic sensor attacks reduces attacked validation loss substantially, and offline validation shows lower four-state RMSE than the kinematic reference on the reported validation file. The result is an estimator that is still close to the filtering structure used in classical observers, but with a learned robustness layer where hand-tuned noise models are weakest.

## Reproducibility Notes

The source code for this work is available on GitHub:

[https://github.com/kslhuy/QCar2\\_Cran](https://github.com/kslhuy/QCar2_Cran)

The main files used for this report are:

- `robustKLnet.py`: model, kinematic predictor, learned update, loss functions, and runtime estimator;
- `robust_kalmanet_train_config.yaml`: active training, model, and augmentation configuration;
- `models/robust_kalmanet_train_history.json`: saved training history used for Table 3;
- `logs/validation/validation_metrics_best.json`: offline validation values used for Table 4;
- `KalmaRobust_Net_Huy.pdf`: source PDF for the result figures rendered into `report_figures/`.

A minimal command sample for reproducing the offline validation and rebuilding this report is:

```
git clone https://github.com/kslhuy/QCar2_Cran.git
cd QCar2_Cran
ROBUST_DIR=Development/multi_vehicle_self_driving_RealQcar/qcar/Observer/KalmaNet/Robust
cd "$ROBUST_DIR"

python validate_robust_kalmanet.py \
  datasets/robust_kalmanet_dataset_V0_20260401_170829.npz \
  --checkpoint models/robust_kalmanet.best_robust.pt \
  --sequence-length 50 \
  --device auto \
  --output logs/validation/validation_metrics_reproduce.json

latexmk -pdf robust_kalmanet_trust_report.tex
```

## References

- [1] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. G. van Sloun, and Y. C. Eldar, “KalmanNet: Neural network aided Kalman filtering for partially known dynamics,” *IEEE Transactions on Signal Processing*, vol. 70, pp. 1532–1547, 2022.
- [2] P. Dahal, S. Mentasti, L. Paparusso, S. Arrigoni, and F. Braghin, “RobustStateNet: Robust ego vehicle state estimation for Autonomous Driving,” *Robotics and Autonomous Systems*, vol. 172, article 104585, 2024.